

연락처 관리 웹 서비스 — TRD (기술 요구사항 정의서) (2차 과제)

문서명	연락처 관리 웹 서비스(2차 과제) TRD
문서 유형	Technical Requirements Document
과제 구분	2차 과제 — FastAPI + DB 연락처 프로그램
버전	v1.0
상태	확정(Baseline)
근거 문서	00 과제목적, 01 구현요구사항, 02 화면정의서, 03 기능정의서, 04 PRD

이 문서의 역할(안내) TRD는 "어떻게 만드는가(How)"만 다룹니다. "무엇을 만드는가(What)"는 01(구현요구사항)·04(PRD)의 몫이고, 이 문서는 그 두 문서의 요구사항을 실제로 동작하는 시스템으로 옮기기 위한 **기술 스택 확정, DB 설계(DDL), 인증 설계, 모듈 설계, 개발·테스트 환경**을 정의합니다. 03(기능정의서)이 이미 파일·함수 단위 설계를 다뤘으므로, 본 문서는 그 설계를 실행 가능한 수준(실제 DDL, 실제 설정값)으로 구체화하는 데 집중합니다.

1. 기술 스택 확정

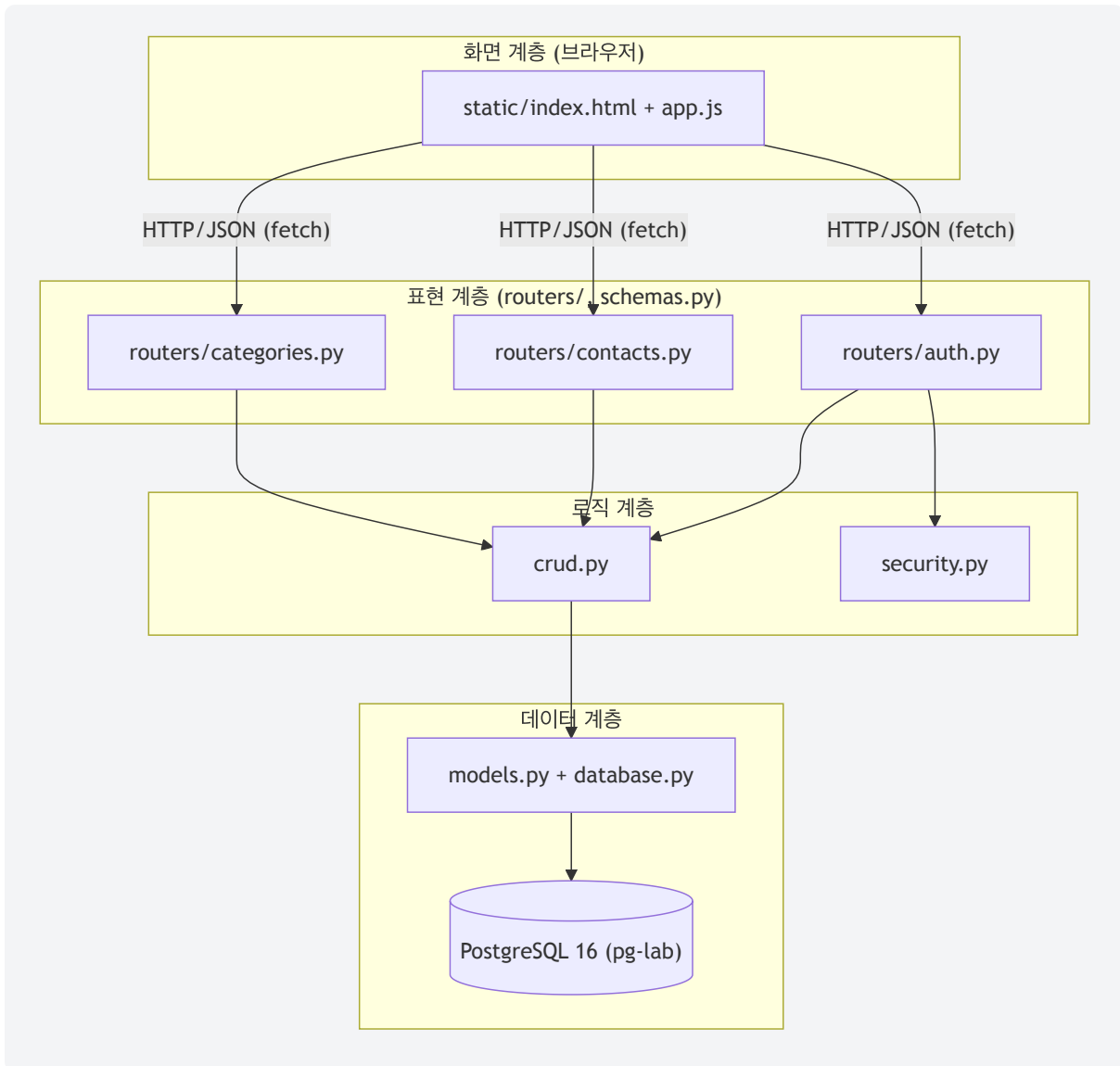
00 문서에서 실제 설치·검증을 마친 조합을 그대로 확정합니다. 버전을 임의로 올리거나 내리지 않습니다.

구성요소	버전	역할
Python	3.12+	실행 환경
FastAPI	0.139.0	웹 프레임워크 (API 서버)
Pydantic	2.13.4	입력 데이터 자동 검증
SQLAlchemy	2.0.51	ORM
psycopg	3.3.4	PostgreSQL 드라이버
pwdlib[argon2]	0.3.0	비밀번호 해싱 (Argon2)
Uvicorn	0.50.0	ASGI 웹 서버
PostgreSQL	16 (Docker, 컨테이너명 <code>pg-lab</code>)	데이터베이스
pytest-playwright	(본 프로젝트에 기설치)	E2E / API 테스트

인증 방식은 00 문서의 결정대로 **세션 쿠키**로 확정합니다 (JWT는 비목표 — 04 문서 §11).

2. 전체 아키텍처

4계층으로 분리하며, 각 계층은 자신의 아래 계층만 호출합니다(계층을 건너뛴 호출 금지).



라우터는 상태 코드를 결정하고, `crud.py` 는 DB 작업만 하며 상태 코드를 모릅니다(03 문서 §1-2 계층 원칙을 그대로 계승).

CORS 미들웨어는 추가하지 않습니다. `main.py` 가 `GET /` 으로 `static/index.html` 을 같은 FastAPI 앱에서 직접 서빙하므로(§3), 화면과 API가 항상 같은 오리진(`127.0.0.1:8000`)에서 동작합니다. 화면을 다른 포트의 별도 서버로 띄우지 않는 한 `CORSMiddleware` 는 불필요하며, 필요하지 않은 설정을 미리 추가하지 않는 것이 이번 과제의 단순함 원칙에도 맞습니다.

3. 프로젝트 파일 구조

03 문서(기능정의서)의 구조를 기술 실행 기준으로 확정합니다.

```

backend/
├── main.py          # FastAPI 앱 생성, 라우터 등록, 정적 파일 마운트
├── database.py      # engine, SessionLocal, Base, get_db()
├── models.py        # User, LoginSession, Category, Contact
├── schemas.py       # Pydantic 입출력 스키마
├── security.py      # hash_password, verify_password
├── crud.py          # DB 작업 함수 (조회/생성/수정/삭제)
├── routers/
│   ├── auth.py      # signup, login, logout, me, get_current_user
│   ├── contacts.py  # 연락처 CRUD 4개
│   └── categories.py # 카테고리 CRUD 4개
├── static/
│   ├── index.html   # SCR-001~003
│   └── app.js        # fetch 호출 + DOM 갱신
└── requirements.txt

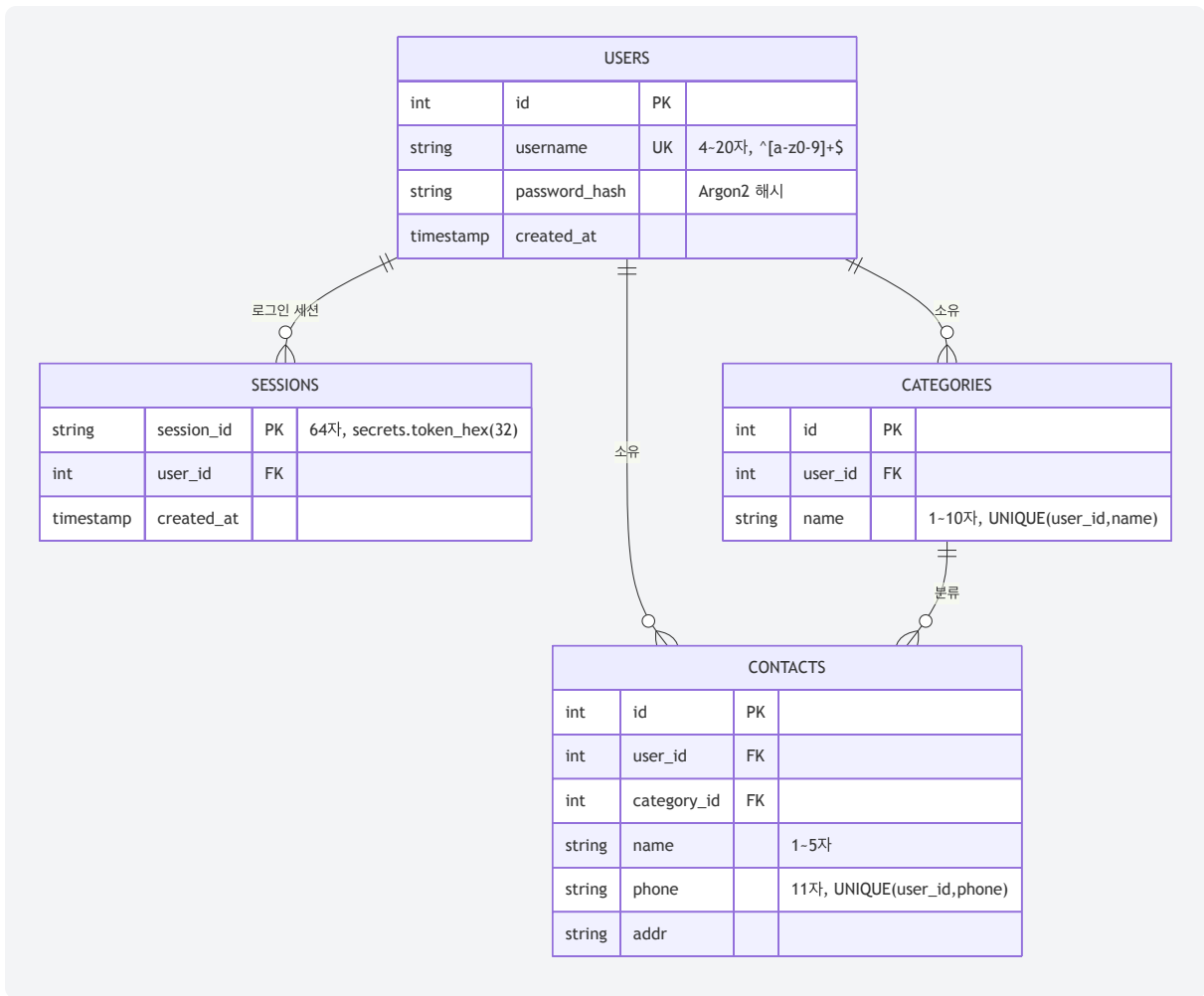
```

backend/CLAUDE.md 에 명시된 "FastAPI 사용" 방침에 따라 백엔드 코드는 backend/ 아래에 위치합니다.

4. DB 설계

4-1. ER 다이어그램

01 문서 §1-1의 관계도를 그대로 반영합니다.



4-2. DDL

01 문서 §1-2의 표를 실제 실행 가능한 PostgreSQL 16 DDL로 확정합니다.

```

CREATE TABLE users (
    id            SERIAL PRIMARY KEY,
    username      VARCHAR(20) NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE sessions (
    session_id CHAR(64) PRIMARY KEY,
    user_id    INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE categories (
    id            SERIAL PRIMARY KEY,
    user_id       INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name          VARCHAR(10) NOT NULL,
    UNIQUE (user_id, name)
);

CREATE TABLE contacts (
    id            SERIAL PRIMARY KEY,
    user_id       INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    category_id   INTEGER NOT NULL REFERENCES categories(id), -- ON DELETE 지정 없음 = 기본
    name          VARCHAR(5) NOT NULL,
    phone         CHAR(11) NOT NULL,
    addr          VARCHAR(255) NOT NULL DEFAULT '',
    UNIQUE (user_id, phone)
);

```

설계 근거

- `username` 에는 형식 검증용 `CHECK` 제약을 걸지 않습니다. 01 문서 §4-2의 "①형식 검증은 Pydantic, ②데이터 검증(중복 등)만 DB/코드가 담당"이라는 2계층 원칙을 DDL에서도 그대로 지키기 위함입니다 — 정규식 규칙이 Pydantic과 DB 두 곳에 흩어지면, 나중에 규칙을 바꿀 때 한쪽을 빠뜨리는 사고가 나기 쉽습니다.
- `users`, `categories` 에는 `ON DELETE CASCADE` 를 걸어, 계정 삭제 시 그 사용자의 세션·카테고리·연락처가 고아 데이터로 남지 않게 합니다. (회원 탈퇴 기능은 01 문서 FR 목록과 04 문서 PR 목록 어디에도 없어 이번 구현 범위 밖이지만, FK 정책은 미리 안전하게 정의해 둡니다.)
- `contacts.category_id` 는 **`ON DELETE` 절을 지정하지 않습니다**. PostgreSQL 기본 동작(`NO ACTION`) 이 "연락처가 남아있는 카테고리는 삭제할 수 없다"를 DB 레벨에서 보장하며, 이것이 01 문서 §3-3 FR-12(사용 중 카테고리 삭제 거부, 409)의 최종 안전망입니다. 실제 409 응답은 `crud.count_contacts_in_category()` 로 애플리케이션 레벨에서 먼저 차단하고(03 문서 FN-010), DB 제약은 그 로직이 누락되었을 때의 마지막 방어선 역할을 합니다.

4-3. SQLAlchemy 2.0 모델 (models.py)

03 문서 §1-3의 명명 규칙(`LoginSession` — SQLAlchemy `Session` 과의 이름 충돌 회피)을 그대로 따릅니다.

```

from datetime import datetime

from sqlalchemy import ForeignKey, String, UniqueConstraint, func
from sqlalchemy.orm import Mapped, mapped_column

from database import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(20), unique=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(nullable=False)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())

class LoginSession(Base):
    __tablename__ = "sessions"

    session_id: Mapped[str] = mapped_column(String(64), primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())

class Category(Base):
    __tablename__ = "categories"
    __table_args__ = (UniqueConstraint("user_id", "name"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
    name: Mapped[str] = mapped_column(String(10), nullable=False)

class Contact(Base):
    __tablename__ = "contacts"
    __table_args__ = (UniqueConstraint("user_id", "phone"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
    category_id: Mapped[int] = mapped_column(ForeignKey("categories.id"), nullable=False)
    name: Mapped[str] = mapped_column(String(5), nullable=False)
    phone: Mapped[str] = mapped_column(String(11), nullable=False)
    addr: Mapped[str] = mapped_column(String(255), nullable=False, default="")

```

앱 시작 시 `Base.metadata.create_all(bind=engine)` 으로 테이블을 자동 생성합니다(교육용 과제 범위이므로 Alembic 마이그레이션은 04 문서 §11 확장 대상에 포함하지 않습니다).

4-4. DB 세션 관리·접속 설정 (database.py)

03 문서 FN-0010이 "접속 문자열은 `postgresql+psycopg://` 로 시작 — 상세는 05 TRD"라고 위임한 부분을 여기서 확정합니다.

```
import os

from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = os.environ["DATABASE_URL"]
# 형식: postgresql+psycopg://{user}:{password}@{host}:{port}/{db}
# 예:   postgresql+psycopg://contact_app:{password}@localhost:5432/contact_app

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

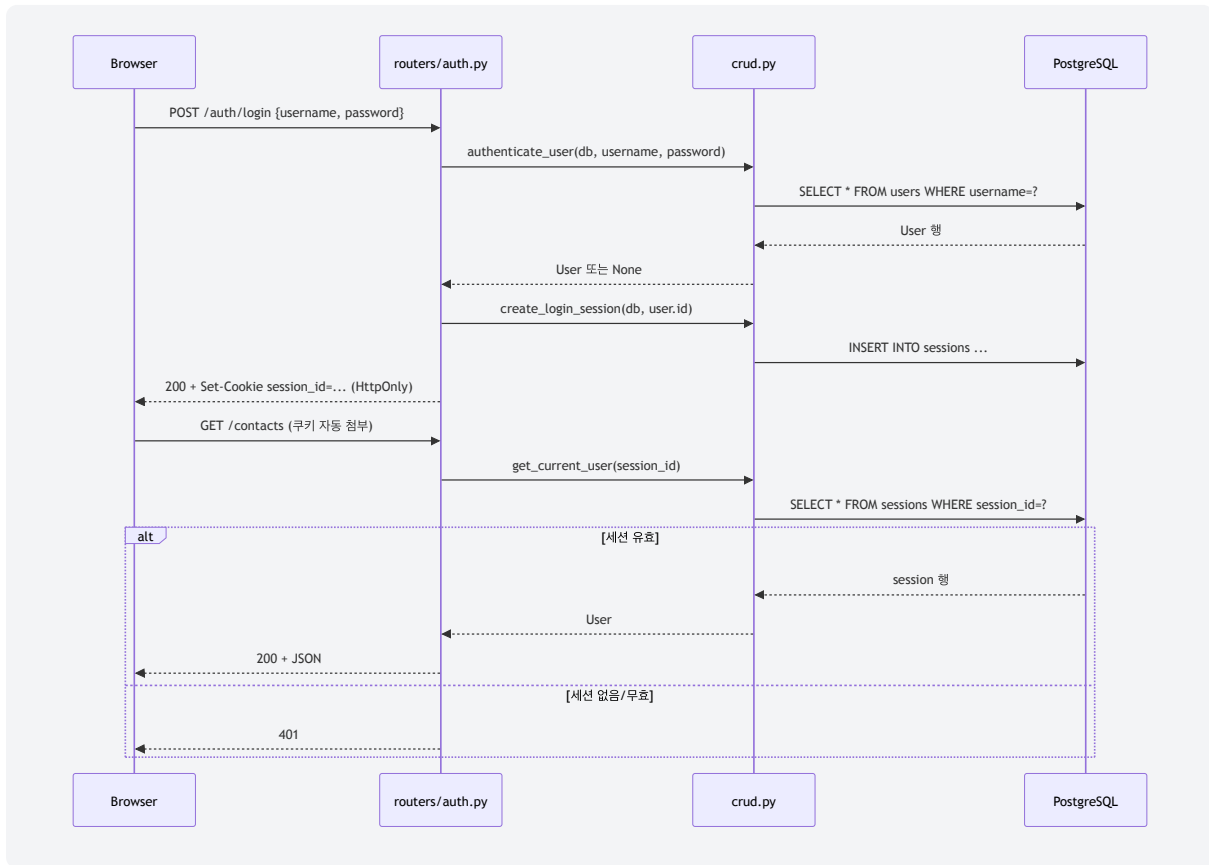
연결 문자열: `DATABASE_URL` 은 `.env` 에 두고 `os.environ` 으로 읽습니다. 실제 비밀번호 값은 CLAUDE.md 규칙(`.env` 직접 수정 금지)에 따라 이 문서에도, 저장소 어디에도 평문으로 남기지 않습니다.

문자 인코딩(NFR-05, 한글이 깨지지 않을 것): PostgreSQL 16 공식 Docker 이미지는 기본 `ENCODING=UTF8` 로 초기화되고, `psycopg3`는 별도 설정 없이 UTF-8을 기본으로 사용하므로 한글 데이터가 추가 설정 없이 안전하게 오갑니다. FastAPI의 JSON 응답도 기본 UTF-8이며, 02 문서 §8이 요구하는 `<meta charset="UTF-8">` 은 `static/index.html` 에서 그대로 지킵니다.

트랜잭션 롤백(01 문서 §6 "처리 도중 오류가 나면 롤백"): `get_db()` 가 예외를 잡으면 `rollback()` 후 다시 던지므로, 라우터가 `HTTPException` 을 일으키기 전에 이미 실행된 `INSERT / UPDATE` 가 있어도 커밋되지 않고 되돌아갑니다. 반쪽짜리 데이터가 남지 않습니다.

5. 인증 설계 (세션 쿠키)

5-1. 로그인 → 보호 API 호출 흐름



5-2. 기술 결정 사항

항목	결정	근거
세션 번호 생성	<code>secrets.token_hex(32)</code> (표준 라이브러리)	00 문서 — 추가 라이브러리 0개
쿠키 옵션	<code>httponly=True</code>	JavaScript가 <code>document.cookie</code> 로 세션 값을 읽지 못하게 하여 XSS로 인한 세션 탈취 방지
쿠키 옵션	<code>samesite="lax"</code>	로컬 개발(127.0.0.1) 환경에서 CSRF 기본 방어, 동일 출처 fetch에는 영향 없음
비밀번호 저장	<code>pwdlib[argon2]</code> 의 <code>hash()</code> / <code>verify()</code>	00 문서 — FastAPI 공식 문서 권장 해시
<code>get_current_user</code> 시그니처	<code>session_id: str None = Cookie(default=None), db: Session = Depends(get_db)</code>	03 문서 FN-007 그대로
로그인 실패 문구	아이디 불명/비밀번호 오류 구분 없이 동일 메시지	01 문서 §3-1 — 계정 존재 여부 노출 방지
데이터 격리	모든 조회 함수의 WHERE 절에 <code>user_id</code> 필수 포함	남의 데이터는 403이 아니라 404 (01 문서 §5-2)

6. API 설계 요약

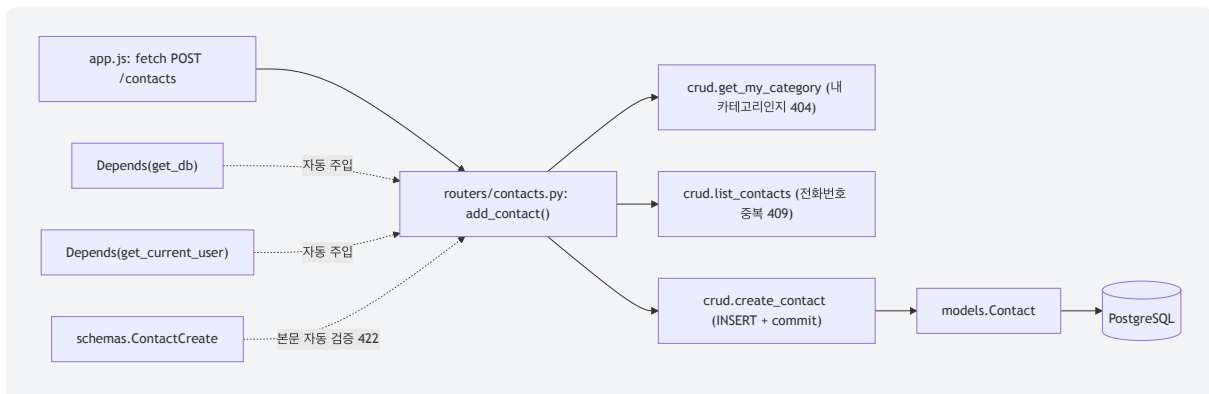
01 문서 §2의 FR 목록을 엔드포인트 계약으로 확정합니다. 상세 요청/응답 본문은 01 문서 §3을 단일 기준(source of truth)으로 따릅니다.

그룹	메서드 · 경로	인증 필요	성공 코드	주요 실패 코드
인증	POST /auth/signup	불필요	201	409, 422
인증	POST /auth/login	불필요	200	401
인증	POST /auth/logout	필요	200	-
인증	GET /auth/me	필요	200	401
연락처	POST /contacts	필요	201	401, 404, 409, 422
연락처	GET /contacts	필요	200	401
연락처	PATCH /contacts/{id}	필요	200	401, 404, 409, 422
연락처	DELETE /contacts/{id}	필요	204	401, 404
카테고리	GET /categories	필요	200	401
카테고리	POST /categories	필요	201	401, 409, 422
카테고리	PATCH /categories/{id}	필요	200	401, 404, 409, 422
카테고리	DELETE /categories/{id}	필요	204	401, 404, 409
화면	GET /	불필요	200	-

모든 오류 응답은 `{"detail": "..."}` 단일 형태로 통일하며(01 문서 §5-2), 어떤 경로로도 `500` 이 발생하지 않아야 합니다(NFR-01).

7. 모듈 설계 (의존성 흐름)

03 문서 §3의 함수 호출 관계를 `Depends` 의존성 주입 관점에서 재정리합니다. "연락처 추가"를 대표 예로 듭니다.



핵심 원칙: 라우터는 접수·인증확인·데이터검증 실패를 상태 코드로 변환만 하고, 실제 DB 작업은 전부 `crud.py` 가 수행합니다. `crud.py` 는 FastAPI를 import하지 않으며 `HTTPException` 을 던지지 않습니다 — 이렇게 해야 나중에

화면을 CLI나 다른 프레임워크로 바꿔도 `crud.py` 를 재사용할 수 있습니다(03 문서 §1-2 계층 분리 원칙).

8. 예외·검증 구현 전략

01 문서 §4-2의 2계층 검증을 코드 레벨로 확정합니다.

계층	구현 위치	실패 시
① 형식 검증	<code>schemas.py</code> 의 Pydantic <code>Field(min_length=..., pattern=...)</code>	FastAPI가 자동으로 422
② 데이터 검증	<code>routers/*.py</code> 에서 <code>crud.py</code> 반환값을 확인 후 <code>raise HTTPException(...)</code>	404 / 409

8-1. 형식 검증 — `schemas.py`

01 문서 §4-1 규칙표를 Pydantic 필드로 그대로 옮깁니다.

```

from pydantic import BaseModel, Field

class SignupIn(BaseModel):
    username: str = Field(min_length=4, max_length=20, pattern=r"^[a-z0-9]+$")
    password: str = Field(min_length=4, max_length=20)

class UserOut(BaseModel):
    id: int
    username: str
    # password_hash는 절대 포함하지 않는다 (03 문서 FN-003)

class ContactCreate(BaseModel):
    name: str = Field(min_length=1, max_length=5)
    phone: str = Field(pattern=r"^010\d{8}$")
    addr: str = ""
    category_id: int

class ContactUpdate(BaseModel):
    name: str | None = Field(default=None, min_length=1, max_length=5)
    phone: str | None = Field(default=None, pattern=r"^010\d{8}$")
    addr: str | None = None
    category_id: int | None = None

class ContactOut(BaseModel):
    id: int
    name: str
    phone: str
    addr: str
    category_id: int
    category_name: str

class CategoryCreate(BaseModel):
    name: str = Field(min_length=1, max_length=10)

```

ContactUpdate 는 모든 필드가 None 기본값이라, 라우터에서 `data.model_dump(exclude_unset=True)` 로 보낸 항목만 갱신할 수 있습니다(03 문서 FN-009 `update_contact`). 입력용(`*Create` / `*Update`)과 출력용(`*Out`) 스키마를 분리해, 사용자가 `id` 를 임의로 지정하거나 응답에 `password_hash` 가 노출되는 사고를 원천 차단합니다(03 문서 §4 FN-003).

8-2. 데이터 검증 — 표현 계층

```
# routers/contacts.py 표준 패턴 (03 문서 FN-011)
@router.post("", response_model=schemas.ContactOut, status_code=201)
def add_contact(
    data: schemas.ContactCreate,
    user: models.User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    if not crud.get_my_category(db, user.id, data.category_id):
        raise HTTPException(404, "해당 카테고리가 없습니다")
    if crud.find_contact_by_phone(db, user.id, data.phone):
        raise HTTPException(409, "이미 등록된 전화번호입니다")
    return crud.create_contact(db, user.id, data)
```

이 패턴을 12개 엔드포인트 전부에 동일하게 적용하며, `crud.py` 함수가 `None` 을 반환하면 라우터가 404로, 중복이 감지되면 409로 변환합니다(01 문서 §5-1 상태 코드 매트릭스와 1:1 대응).

9. 개발·실행 환경

9-1. PostgreSQL (Docker)

```
# docker-compose.yml (backend/ 기준)
services:
  pg-lab:
    image: postgres:16
    container_name: pg-lab
    environment:
      POSTGRES_USER: contact_app
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: contact_app
    ports:
      - "5432:5432"
    volumes:
      - pg_lab_data:/var/lib/postgresql/data

volumes:
  pg_lab_data:
```

비밀번호 등 민감정보는 `.env` 에 두고 `docker-compose.yml` 은 `${POSTGRES_PASSWORD}` 로만 참조합니다.

`.env` 는 CLAUDE.md 규칙에 따라 이 TRD 작성 범위에서 직접 생성·수정하지 않으며, 실제 구현 단계에서 개발자가 로컬에 만듭니다.

9-2. Python 환경

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload
```

```
# requirements.txt (예정 구성)
fastapi==0.139.0
pydantic==2.13.4
sqlalchemy==2.0.51
psycopg[binary]==3.3.4
pewdlib[argon2]==0.3.0
uvicorn[standard]==0.50.0
```

서버 기동 후 `http://127.0.0.1:8000/docs` 가 열리면 FN-001~002(DB 연결·테이블 생성)까지 정상입니다(03 문서 §5 1단계 완료 기준).

10. 테스트 전략

프로젝트에 이미 세팅된 `pytest-playwright` 환경(`tests/` 디렉터리, 프로젝트 루트 `.venv`)을 그대로 활용합니다. 01 문서의 FR-04 문서의 AC를 테스트 케이스로 직접 변환하는 것을 목표로 합니다(Karpathy 행동지침 — Goal-Driven Execution).

10-1. 테스트 파일 구성 (제안)

```
tests/
├── test_auth.py           # FR-01~04 → AC-01~04, AC-15
├── test_contacts.py      # FR-05~08 → AC-05~09, AC-13, AC-14
├── test_categories.py    # FR-09~12 → AC-10~12
└── test_persistence.py   # FR 전체 → AC-16 (서버 재시작 후 데이터 유지)
```

10-2. 두 계층의 테스트 도구

대상	도구	이유
API 계약 (FR-01~13)	<code>playwright</code> 의 <code>APIRequestContext</code> (<code>page.request</code> 또는 <code>pytest-playwright</code> 의 <code>request</code> 픽스처)	화면 없이 상태 코드·JSON 본문을 직접 검증 — 01 문서의 401/404/409/422 매트릭스 검증에 적합
화면 흐름 (SCR-001~003)	<code>pytest-playwright</code> 의 <code>page</code> 픽스처	02 문서의 화면 전이(로그인→관리 화면)를 실제 브라우저로 검증

10-3. 데이터 격리 테스트의 핵심 패턴 (AC-13, AC-14)

계정 2개(A, B)를 만들어야 검증되는 요구사항이므로, 브라우저 컨텍스트를 2개 분리해 각자 로그인시킵니다.

```
def test_사용자간_데이터_격리(browser):
    ctx_a = browser.new_context()
    ctx_b = browser.new_context()
    # ctx_a로 A 가입/로그인 + 연락처 등록
    # ctx_b로 B 가입/로그인
    # ctx_b의 request로 A의 연락처 id에 GET/DELETE 시도 → 404 확인
```

11. 요구사항 추적표 (01 문서 §10 TRD 대응 확정)

요구 ID	본 문서 대응 섹션
FR-01~04	§5 인증 설계, §7 모듈 설계 (<code>routers/auth.py</code> , <code>get_current_user</code>)
FR-05~08	§6 API 설계, §7 모듈 설계 (<code>crud.create_contact</code> 등)
FR-09~12	§6 API 설계, §4-2 DDL의 <code>ON DELETE</code> 정책 (사용 중 카테고리 삭제 방어)
FR-13	§3 프로젝트 파일 구조 (<code>static/index.html</code> , <code>main.py</code>)
NFR-01 (500 금지)	§8-2 데이터 검증 — 표현 계층
NFR-02 (계층별 파일 분리)	§2 전체 아키텍처, §3 프로젝트 파일 구조
NFR-03 (Argon2 해시)	§5-2 기술 결정 사항
NFR-04 (전부 로그인 필수 + <code>user_id</code> 격리)	§5-2 데이터 격리, §7 모듈 설계
NFR-05 (한글 UTF-8)	§4-4 DB 세션 관리·접속 설정
NFR-06 (<code>/docs</code> 자동 문서)	§9-2 Python 환경 (FastAPI 기본 동작, 별도 비활성화 없음)

12. 이번 TRD에서 다루지 않는 것

04 문서 §2-2(비목표)와 동일하게, 다음은 이번 기술 설계 범위에서 제외합니다.

- Alembic 등 DB 마이그레이션 도구 (테이블은 `create_all` 로 1회 생성)
- HTTPS·실서버 배포 설정 (로컬 개발 환경 127.0.0.1 한정)
- JWT 인증 (세션 쿠키로 확정 — §5)
- CI/CD 파이프라인 구성

